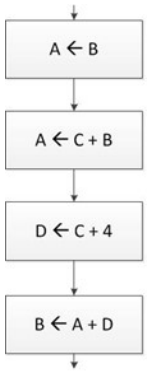
COMPUTER ARCHITECTURE

Hamid Sarbazi-Azad
Department of Computer Engineering
Sharif University of Technology,
Tehran, Iran

Control Unit

Control Unit is responsible for correct operation of all units and components of a digital system.

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

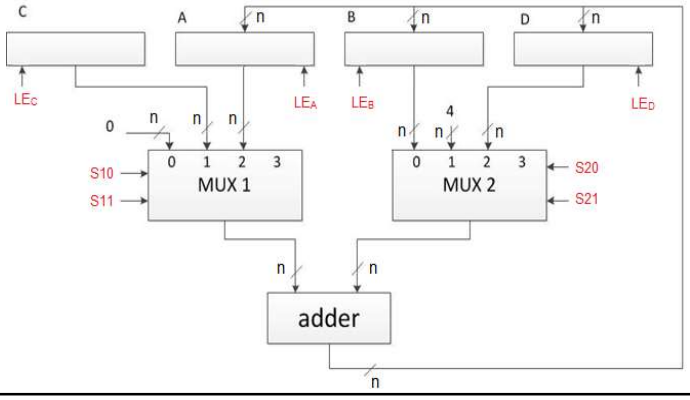


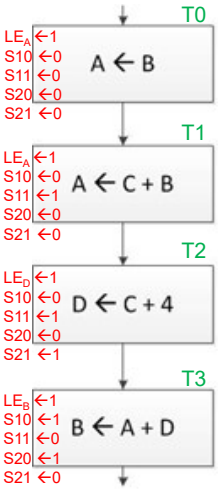
Control Unit

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

We need to deign the data path first, and show all control signals on it.

→ Then, we indicate the activity of each control signal to execute each box of the flowchart.





Control Unit

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.

We need to deign the data path first, and show all control signals on it.

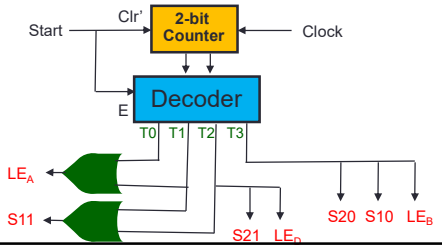
→ Then, we indicate the activity of each control signal to execute each box of the chart.

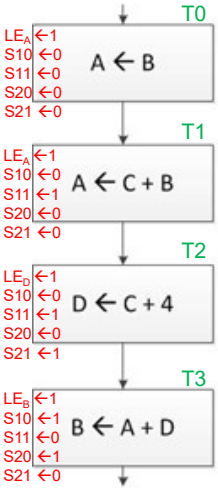
→ Now, we can write down the logic of each control signal as:

$LE_A = T0 + T1$ $LE_B = T3$ $LE_D = T2$

$S10 = S20 = T3$ $S11 = T1 + T2$ $S21 = T2$

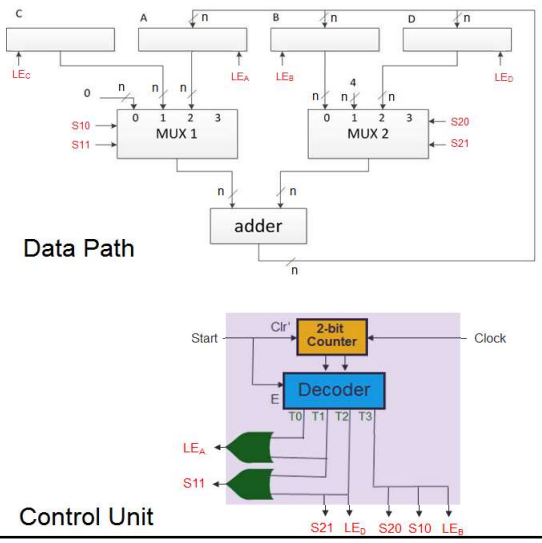
→ So the control unit can be designed as:





Control Unit

Example: Assume the following chart (part of a full chart for a digital system). A, B, C and D are n-bit registers. Design a circuit to implement the char.



Complete System

Control Unit

Control units (or control circuits) can be categorized in:

- Hardwired control units
  - Direct Method
  - Sequencer Based Method
  - Coded State Method

Pros: Low hardware overhead, higher speed
Cons: Design inflexibility

- Microprogrammed control units

Pros: Design flexibility
Cons: Large hardware overhead, lower speed

Control Unit: Hardwired Control Circuits

Direct Method:

In **direct method**, we directly use the system flowchart to design its control circuit.

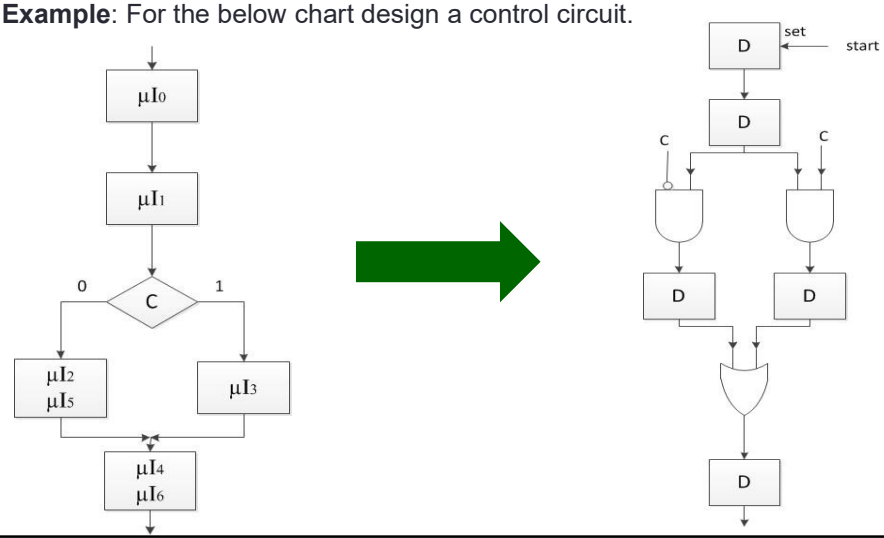
To do so:

1. Each computational/transfer box (or micro-instruction box) is replaced by a D flip-flop
2. Each condition box is replaced by two AND gates
3. Junctions are replaced by OR gates

Control Unit: Hardwired Control Circuits

Direct Method:

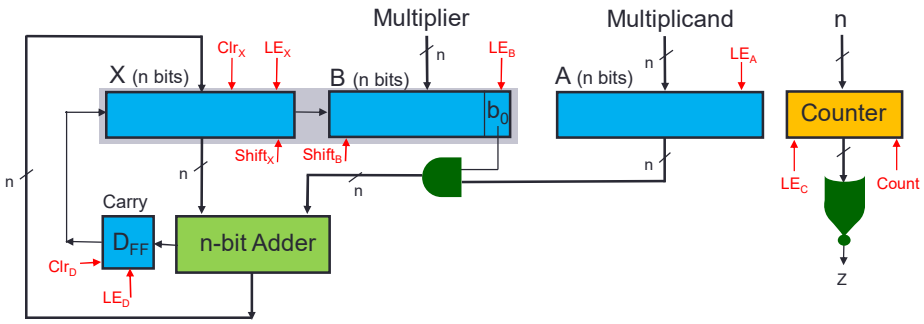
Example: For the below chart design a control circuit.



Control Unit: Hardwired Control Circuits

Direct Method:

Example: Design the control circuit of the Add & Shift Multiplier.
→ Let us first indicate all control signals on data path.

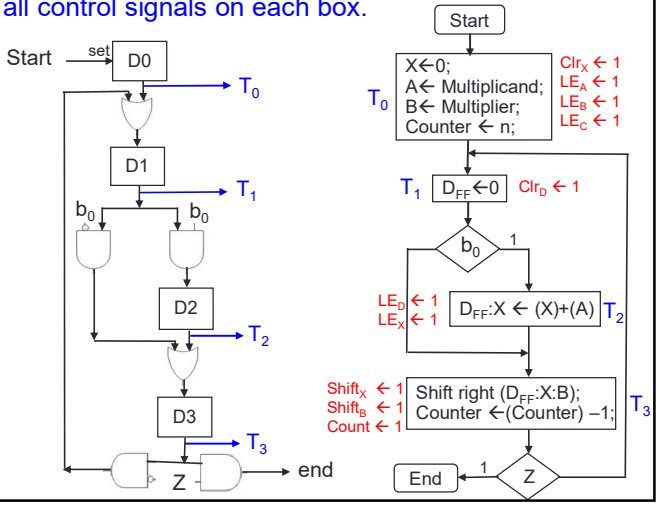


Control Unit: Hardwired Control Circuits

Direct Method:

Example: Design the control circuit of Add & Shift Multiplier.
→ Indicate activity of all control signals on each box.
→ Produce control signals

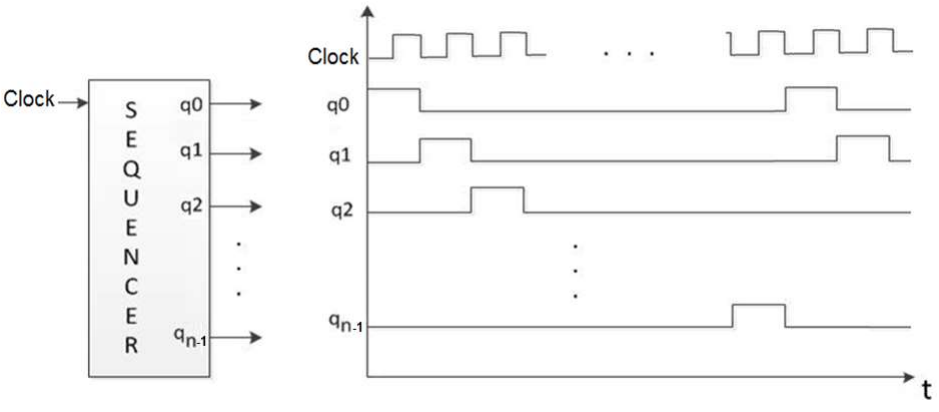
$LE_A = T_0$
$LE_B = T_0$
$LE_C = T_0$
$Clr_X = T_0$
$Clr_D = T_1$
$LE_X = T_2$
$LE_D = T_2$
$Shift_B = T_3$
$Shift_X = T_3$
$Count = T_3$



Control Unit: Hardwired Control Circuits

Sequencer Based Method:

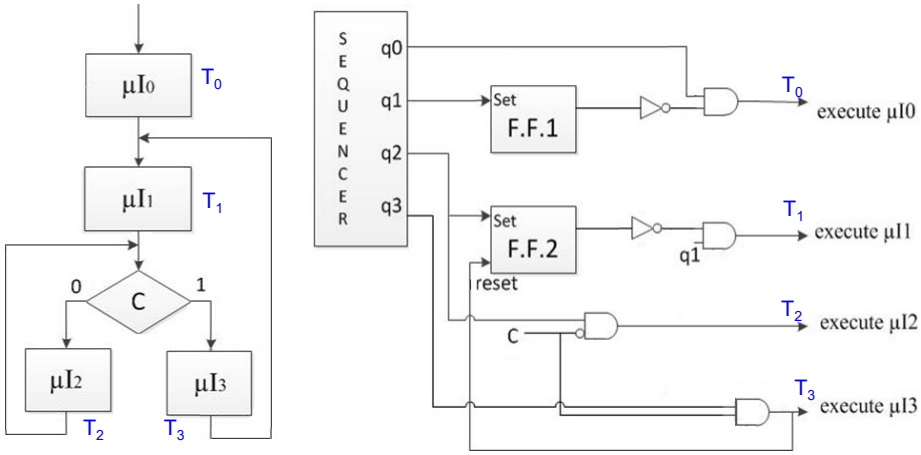
A n-tap Sequencer is a sequential component that has n output lines to generate n consecutive pulses as shown in the figure.



Control Unit: Hardwired Control Circuits

Sequencer Based Method:

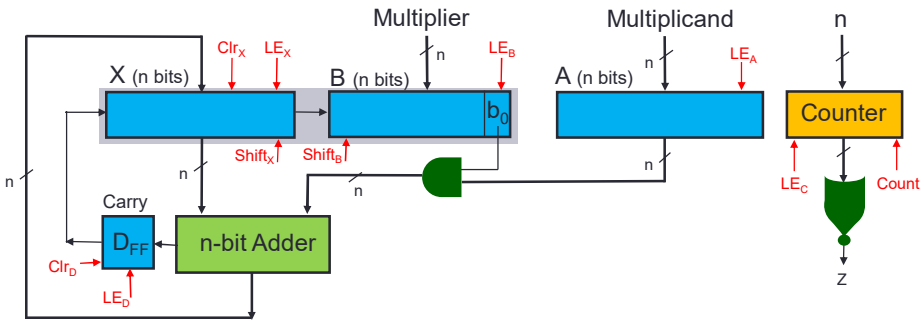
Example: For the below chart design a control circuit.



Control Unit: Hardwired Control Circuits

Sequencer Based Method:

Example: Design the control circuit of the Add & Shift Multiplier.

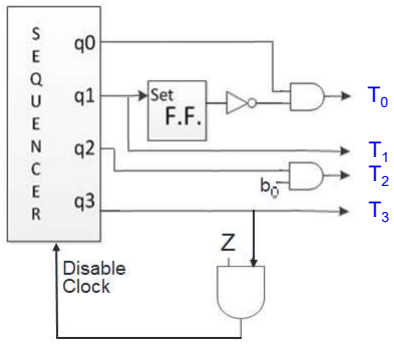


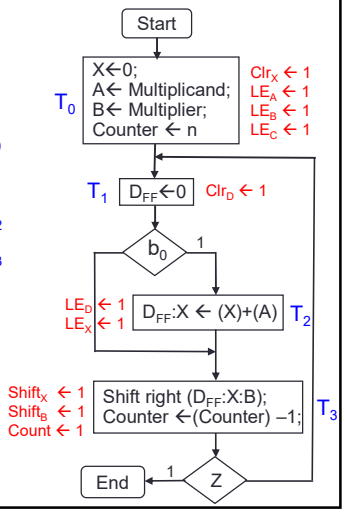
Control Unit: Hardwired Control Circuits

Sequencer Based Method:

Example: Design the control circuit of Add & Shift Multiplier.

$LE_A = T_0$
$LE_B = T_0$
$LE_C = T_0$
$Clr_X = T_0$
$Clr_D = T_1$
$LE_x = T_2$
$LE_D = T_2$
$Shift_B = T_3$
$Shift_X = T_3$
$Count = T_3$





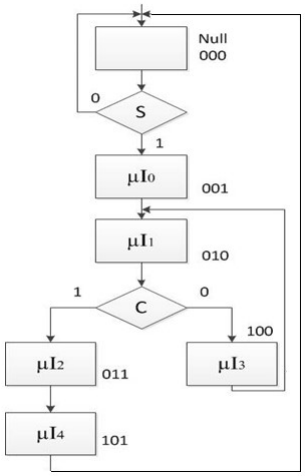
Control Unit: Hardwired Control Circuits

Coded State Method:

Each state shows a box. We need $\log_2 n$ flip-flops for n boxes.

Example: Design the control circuit of the given flowchart.

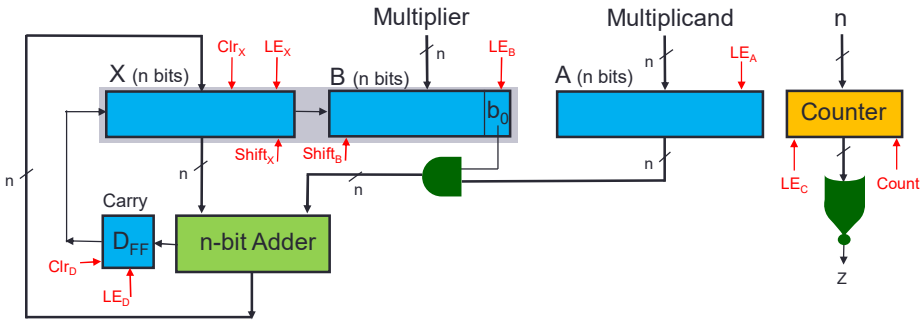
	Next State			
Present State	**CS=00**	**CS=01**	**CS=10**	**CS=11**
000	000	001	000	001
001	010	010	010	010
010	100	100	011	011
011	101	101	101	101
100	010	010	010	010
101	000	000	000	000



Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

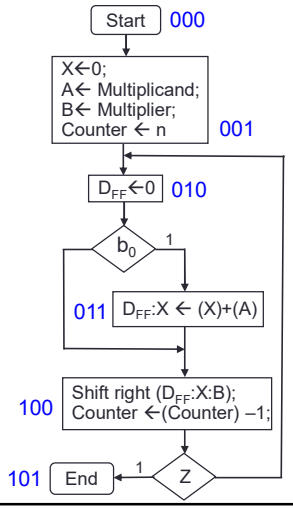


Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

	Next State				
		S = 1			
Present State	S = 0	b_0Z =00	b_0Z =01	b_0Z =10	b_0Z =11
000	000	001	001	001	001
001	000	010	010	010	010
010	000	100	100	011	011
011	000	100	100	100	100
100	000	010	101	010	101
101	000	101	101	101	101

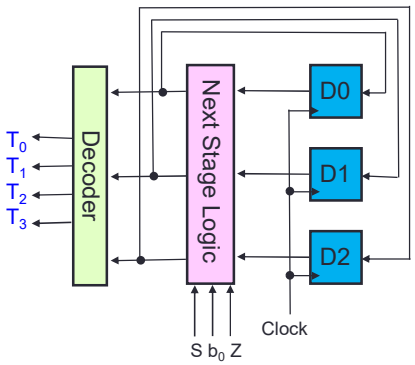


Control Unit: Hardwired Control Circuits

Coded State Method:

Example: Design the control circuit of Add & Shift Multiplier.

	Next State				
		S = 1			
Present State	S = 0	b_0Z =00	b_0Z =01	b_0Z =10	b_0Z =11
000	000	001	001	001	001
001	000	010	010	010	010
010	000	100	100	011	011
011	000	100	100	100	100
100	000	010	101	010	101
101	000	101	101	101	101

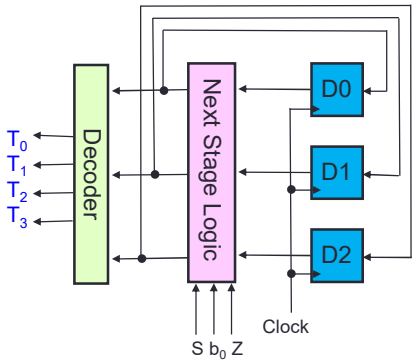


Control Unit: Hardwired Control Circuits

Coded State Method:

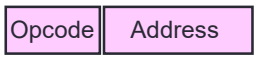
Example: Design the control circuit of Add & Shift Multiplier.

$LE_A = T_0$
$LE_B = T_0$
$LE_C = T_0$
$Clr_X = T_0$
$Clr_D = T_1$
$LE_x = T_2$
$LE_D = T_2$
$Shift_B = T_3$
$Shift_X = T_3$
$Count = T_3$



Control Unit: Hardwired Control Circuits

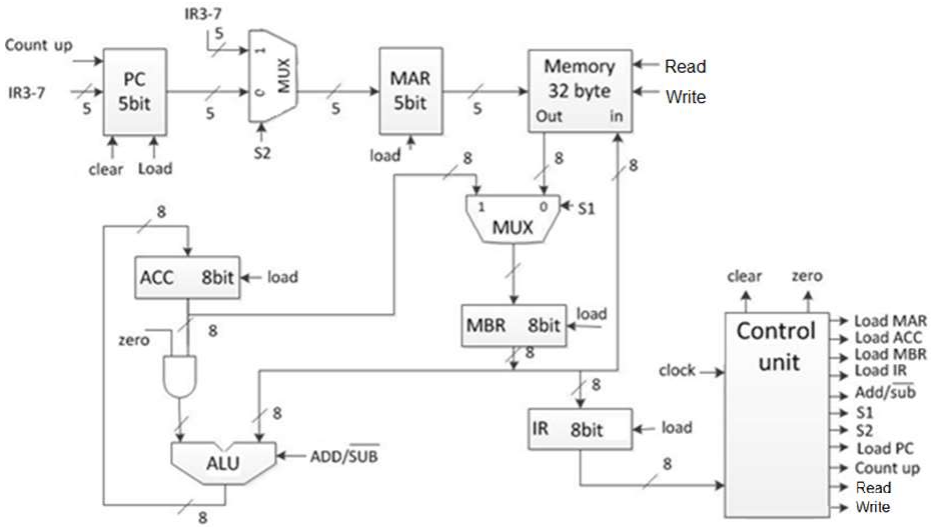
Example: Design the control unit of a simple computer defined below.
- One-address machine
- 32 bytes main memory (word = byte = addressable unit)
- Direct and implicit addressing modes
- 8-bits instructions (3-bit Opcode and 5-bit Address)



Opcode	Mnemonic	Operation
000	LDACC	ACC ← ($M_{address}$);
001	STACC	$M_{address}$ ← (ACC);
010	ADD	ACC ← (ACC)+($M_{address}$);
011	SUB	ACC ← (ACC) - ($M_{address}$);
100	JUMP	PC ← Address;
101	JACCNZ	if (ACC) ≠ 0 then PC ← Address;
110	JACCNEG	if (ACC) < 0 then PC ← Address;
111	NOP	No Operation

Control Unit: Hardwired Control Circuits

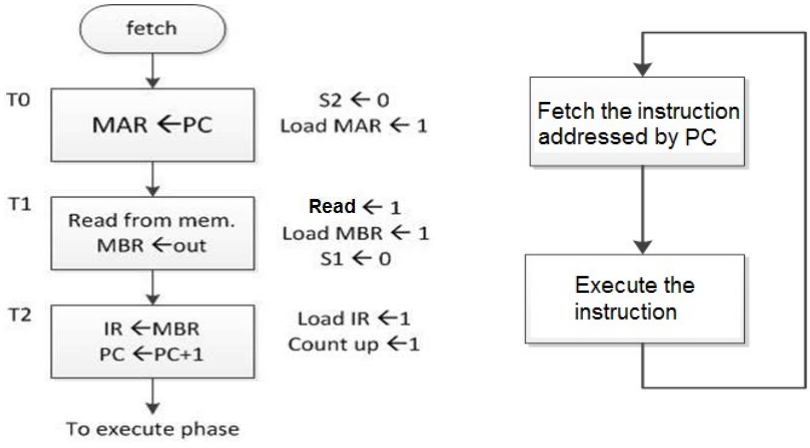
Design Example: Data path and control signals of the given computer.



Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

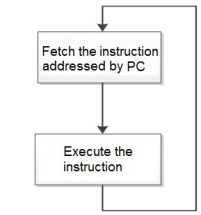
The sub-flowchart of Fetch phase is as follows:

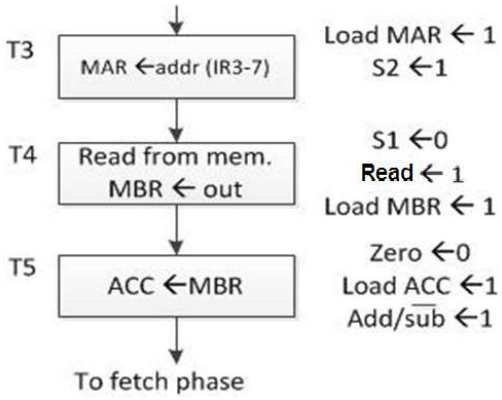


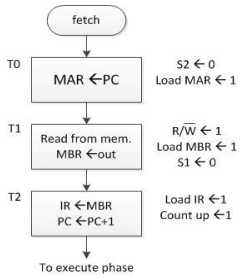
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for LDACC:



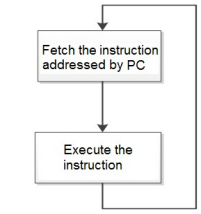


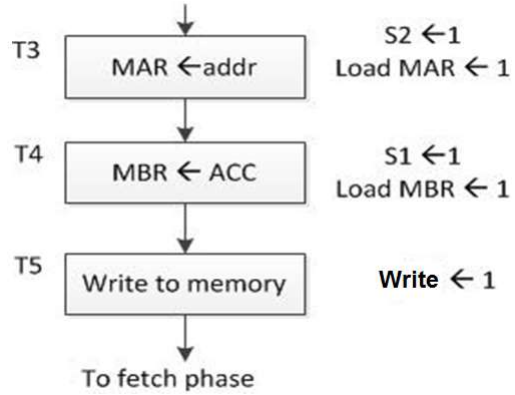


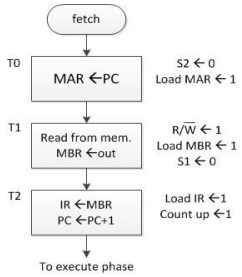
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for STACC:



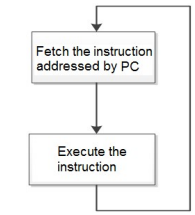


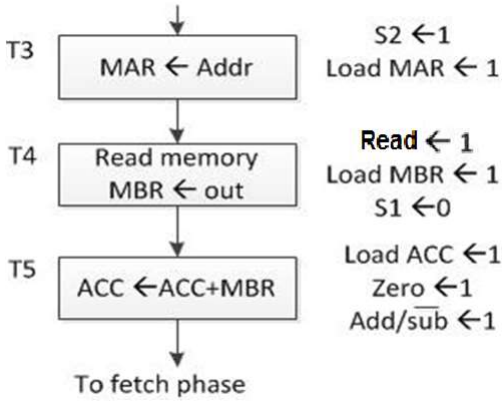


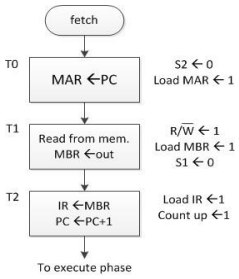
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for ADD:



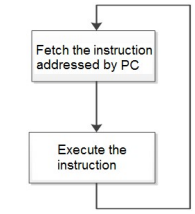


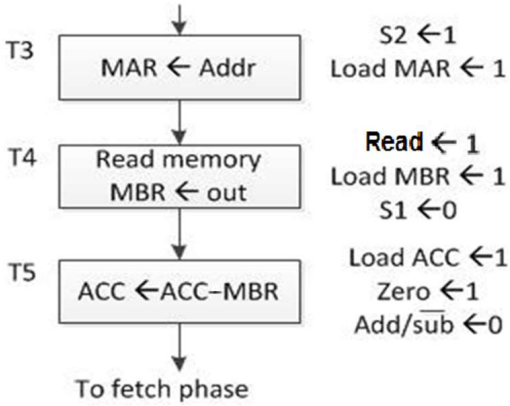


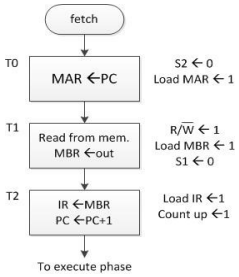
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for SUB:



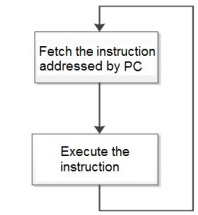


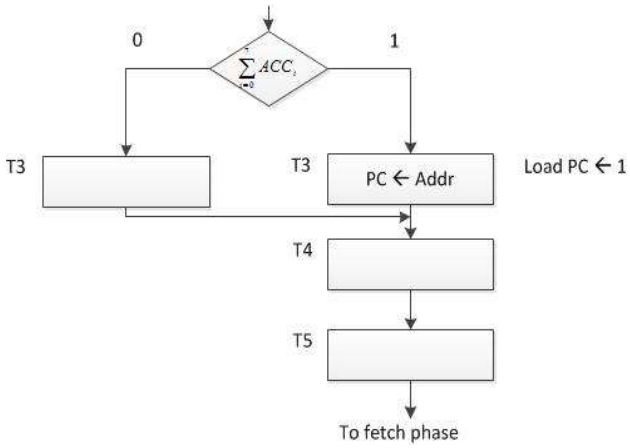


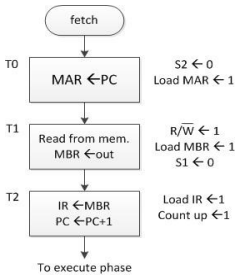
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JACCNZ:



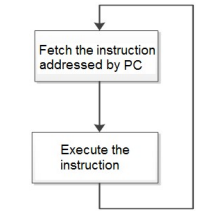


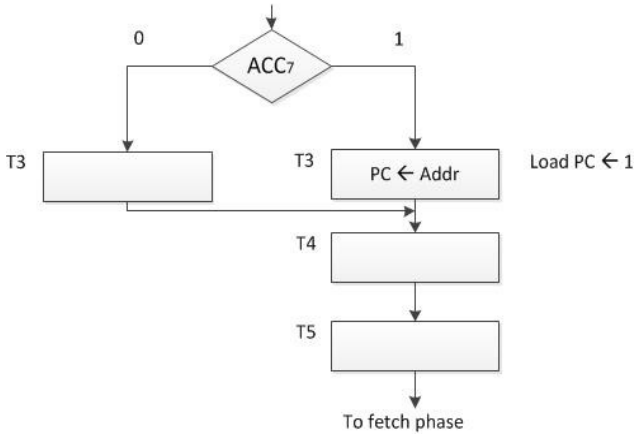


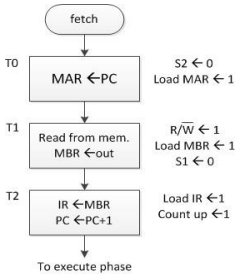
Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JACCNEG:



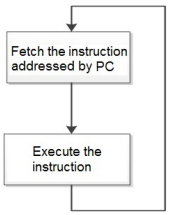


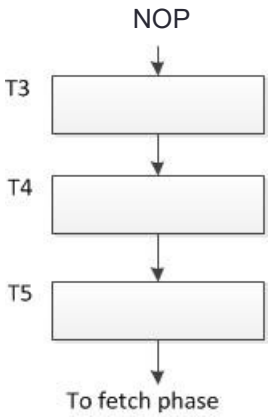


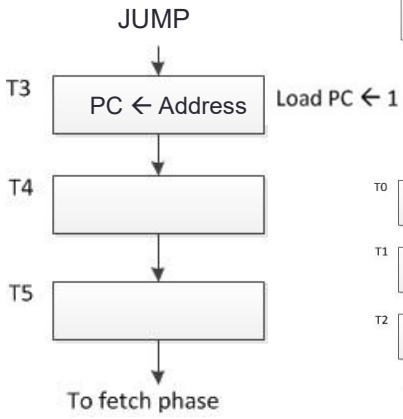
Control Unit: Hardwired Control Circuits

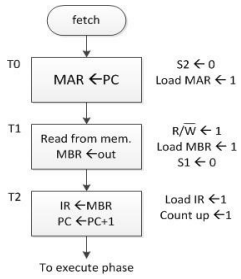
Design Example: Main hardware flow chart.

The sub-flowchart of Execute phase for JUMP and NOP:









Control Unit: Hardwired Control Circuits

Design Example: Main hardware flow chart.

Count up = T_2

Load PC = JUMP.T_3 + JACCNZ.$\sum_{i=0}^{7} ACC_i$.T_3 + JACCNEG.ACC_7.T_3

Load MBR = T_1 + LDACC.T_4 +STACC.T_4 + ADD.T_4 + SUB.T_4

Load MAR = T_0 + LDACC.T_3 +STACC.T_3 + ADD.T_3 + SUB.T_3

Load ACC = LDACC.T_5 + ADD.T_5 + SUB.T_5

Load IR = T_2

S1 = STACC.T_4

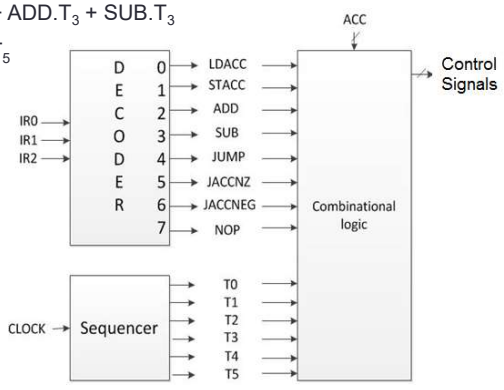
S2 = LDACC.T_3 + ADD.T_3 + SUB.T_3

Read = T1 + T4.(LDACC + ADD + SUB)

Write = STACC.T_5

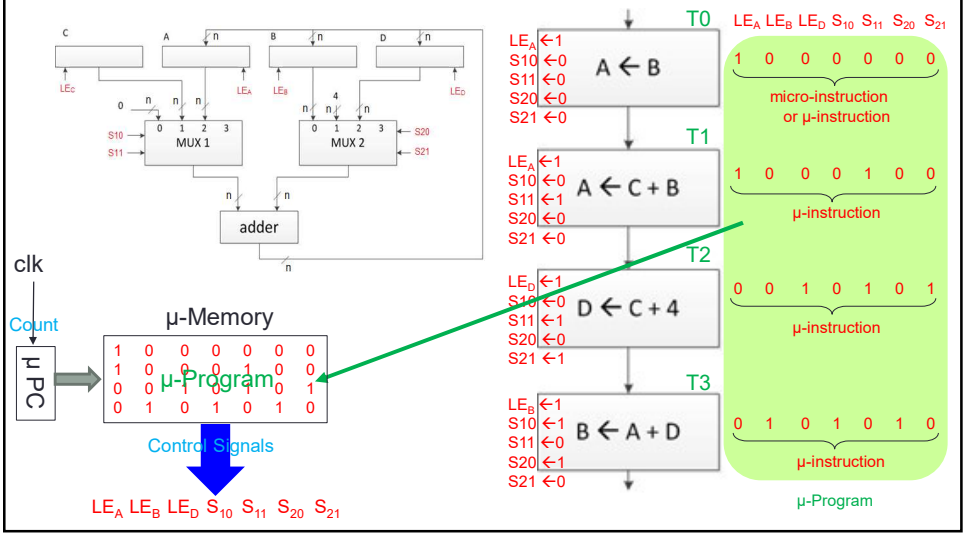
Zero = ADD.T_5 + SUB.T_5

Add/$\overline{\text{Sub}}$ = LDACC.T_5 + ADD.T_5



Control Unit: Microprogrammed CU

Example: Assume the following chart (part of a full chart for a digital system).
A, B, C and D are n-bit registers. Design a circuit to implement the char.



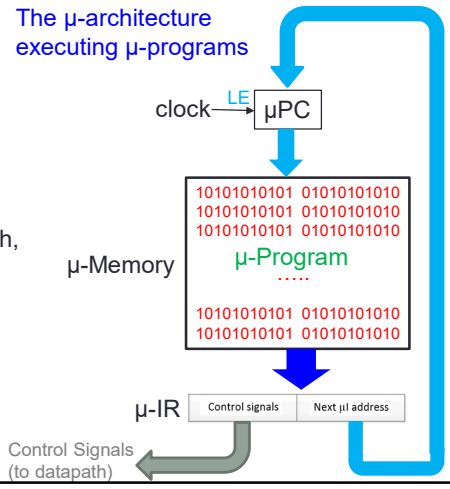
Control Unit: Microprogrammed CU

In order to be able to implement flowcharts with branches, some early micro-instruction formats were proposed.

Micro-instruction format

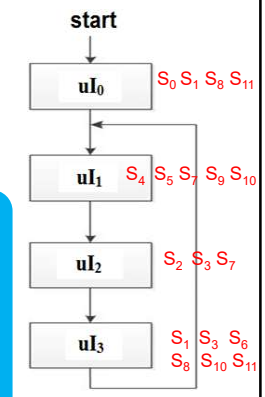
For each µ-Instruction:
1. pass **Control signals** to the datapath,
2. µPC ← **Next µI address**;

The µ-architecture executing µ-programs



Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart (S_0-S_{11} control signals).

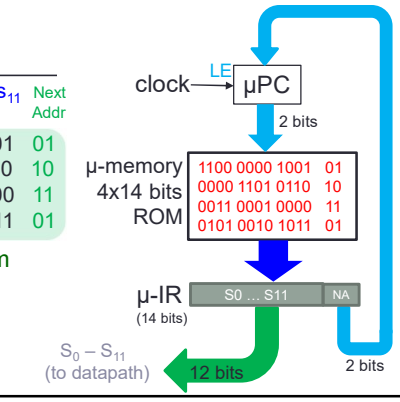


μ-instruction format

μ Address	μ Instruction S_0-S_3	S_4-S_7	S_8-S_{11}	Next Addr
00	1100	0000	1001	01
01	0000	1101	0110	10
10	0011	0001	0000	11
11	0101	0010	1011	01

μ-program

The μ-architecture executing μ-programs



Control Unit: Microprogrammed CU

In order to be able to implement flowcharts with conditional branches, new micro-instruction formats were proposed.

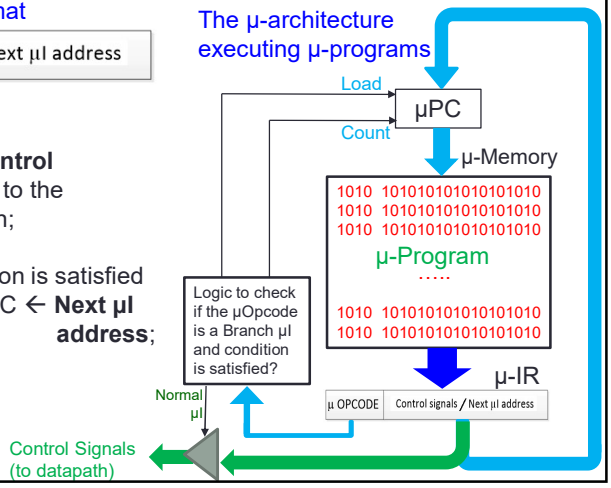
Micro-instruction format

For each μ Instruction:

μ Opcode = 00..0 → pass **Control signals** to the datapath;
(Normal μI)

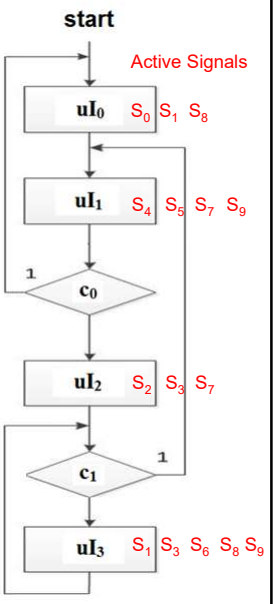
μ Opcode ≠ 00..0 → if condition is satisfied then μPC ← **Next μI address**;
(Branch μI)

The μ-architecture executing μ-programs



Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart.



μ-Instruction Format

Micro-Opcode	Control signals / Next Address
2 bits	10 bits

μ Opcode	Operation
00	Normal μI: Pass S_0-S_9 to datapath
01	If c_0 then μPC ← Next Address
10	If c_1 then μPC ← Next Address
11	μPC ← Next Address

μ Address	μ Instruction
0000	00 1100000010
0001	00 0000110101
0010	01 0000000000
0011	00 0011000100
0100	10 0000000001
0101	00 0101001011
0110	11 0000000100

μ-program

Control Unit: Microprogrammed CU

Example: Implement a microprogrammed control unit for the given flowchart.

μ-Instruction Format

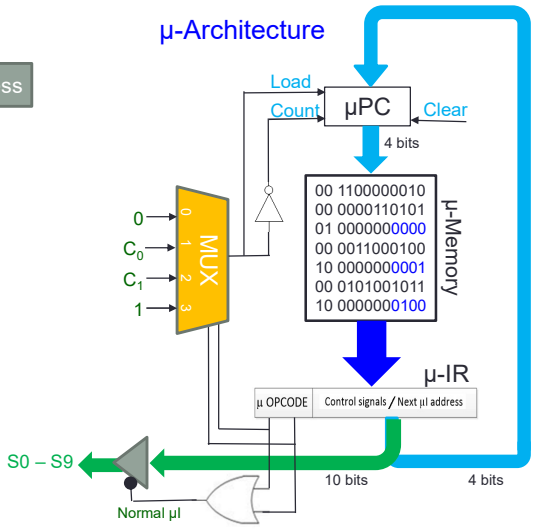
μ Opcode	Control signals / Next Address

μ Opc	Operation
00	Normal μI: S_0-S_9 to datapath
01	If c_0 then μPC ← Next Addr
10	If c_1 then μPC ← Next Addr
11	μPC ← Next Address

μ Address	μ Instruction
0000	00 1100000010
0001	00 0000110101
0010	01 0000000000
0011	00 0011000100
0100	10 0000000001
0101	00 0101001011
0110	10 0000000100

μ-program

μ-Architecture

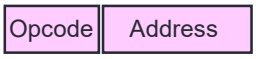


Control Unit: Microprogrammed CU

Design Example: Design the control unit of a simple computer defined below.

- One-address machine
- 32 bytes main memory (word = byte = addressable unit)
- Direct and implicit addressing modes
- 8-bits instructions (3-bit Opcode and 5-bit Address)

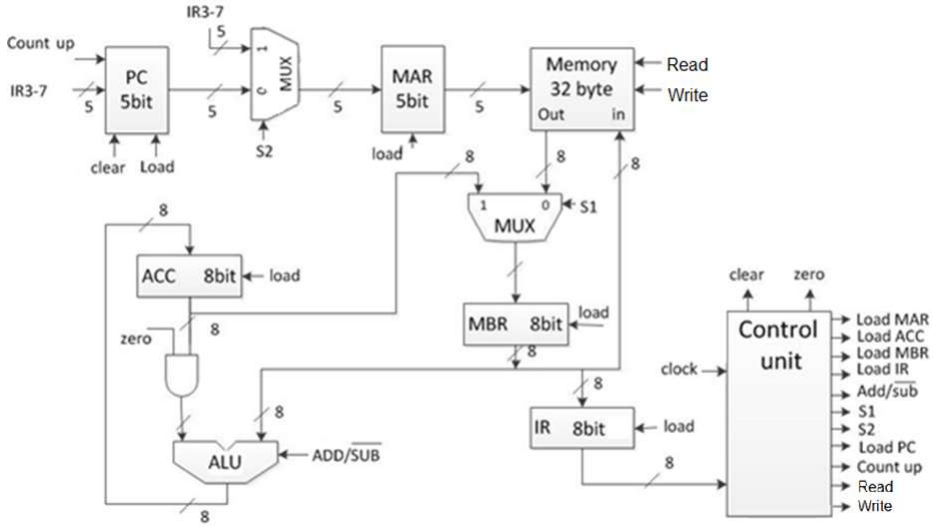
Instruction Format



Opcode	Mnemonic	Operation
000	LDACC	ACC ← ($M_{address}$);
001	STACC	$M_{address}$ ← (ACC);
010	ADD	ACC ← (ACC)+($M_{address}$);
011	SUB	ACC ← (ACC) - ($M_{address}$);
100	JUMP	PC ← Address;
101	JACCNZ	if (ACC) ≠ 0 then PC ← Address;
110	JACCNEG	if (ACC) < 0 then PC ← Address;
111	NOP	No Operation

Control Unit: Microprogrammed CU

Design Example: Data path and control signals of the given computer.



Control Unit: Microprogrammed CU

Control signals (12 signals):

$count_{PC}$, $load_{PC}$, $load_{MBR}$, $load_{MAR}$, $load_{ACC}$, $load_{IR}$, S_1, S_2, Rd, Wr, Zero, $A/\overline{S}$

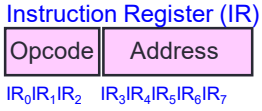
Looking at the flowchart, we can see the following conditional boxes (5 boxes):

(ACC) ≠ 0, (ACC)<0,

and 3 conditions to check the three OPCODE's bits after the fetch phase, that is:

$IR_0 = 1$, $IR_1 = 1$, $IR_2 = 1$

Instruction Register (IR)

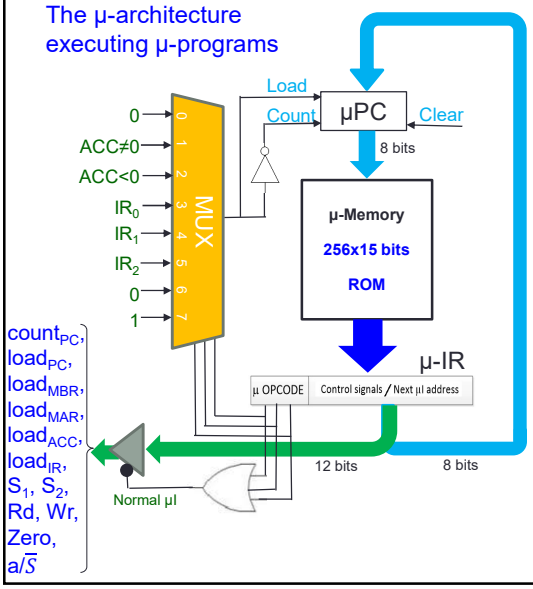


μ-Instruction Format

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Control Unit: Microprogrammed CU

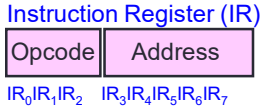
The μ-architecture executing μ-programs



μ-Instruction Format

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Register (IR)



Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	1	0	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011	@JUMP-JACCNZ-JACCNEG-NOP	011												
	00000100	@ADD-SUB	100												
	00000101	@STACC	101												
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011	@JUMP-JACCNZ-JACCNEG-NOP	011												
	00000100	@ADD-SUB	100												
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	0	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011	@JUMP-JACCNZ-JACCNEG-NOP	011												
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101	@SUB	101												
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011	@JUMP-JACCNZ-JACCNEG-NOP	011												
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101		101	0	0	0	0	0	0	0	0	1	0	1	0
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010010		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010011		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010100		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011		011	0	0	0	0	0	0	0	1	0	1	1	0
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101		101	0	0	0	0	0	0	0	1	0	0	1	0
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010010		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010011		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010100		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0
	00010110	@JACCNEG-NOP													
	00010111	@JACCNZ													
JUMP	00011000		000	0	1	0	0	0	0	0	0	0	0	0	0
	00011001		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011		011	0	0	0	0	0	0	0	1	0	1	1	0
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101		101	0	0	0	0	0	0	0	1	0	0	1	0
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010010		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010011		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010100		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0
	00010110	@JACCNEG-NOP													
	00010111		101	0	0	0	0	0	0	0	1	1	0	1	0
JUMP	00011000		000	0	1	0	0	0	0	0	0	0	0	0	0
	00011001		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	A/$\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011		011	0	0	0	0	0	0	0	1	0	1	1	0
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101		101	0	0	0	0	0	0	0	1	0	0	1	0
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010010		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010011		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010100		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0
	00010110	@JACCNEG-NOP													
	00010111		101	0	0	0	0	0	0	0	1	1	0	1	0
JUMP	00011000		000	0	1	0	0	0	0	0	0	0	0	0	0
	00011001		111	0	0	0	0	0	0	0	0	0	0	0	0
JACCNZ	00011010		001	0	0	0	0	0	0	0	**1**	**1**	**0**	**0**	**0**
	00011011		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

Control Unit: Microprogrammed CU

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	A/$\overline{S}$
FETCH	00000000		000	0	0	0	1	0	0	0	1	0	0	0	0
	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011		011	0	0	0	0	0	0	0	1	0	1	1	0
	00000100		100	0	0	0	0	0	0	0	0	1	1	0	1
	00000101		101	0	0	0	0	0	0	0	0	1	0	1	0
LDACC	00000110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001000		000	0	0	0	0	1	0	0	0	0	0	0	1
	00001001		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001010		000	0	0	1	1	0	0	1	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001100		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001101		101	0	0	0	0	0	0	0	1	0	0	1	0
ADD	00001110		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001111		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010000		000	0	0	0	0	1	0	0	0	0	0	1	1
	00010001		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010010		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010011		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010100		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0
	00010110		100	0	0	0	0	0	0	0	1	1	1	0	0
	00010111		101	0	0	0	0	0	0	0	1	1	0	1	0
JUMP	00011000		000	0	1	0	0	0	0	0	0	0	0	0	0
	00011001		111	0	0	0	0	0	0	0	0	0	0	0	0
JACCNZ	00011010		001	0	0	0	0	0	0	0	**1**	**1**	**0**	**0**	**0**
	00011011		111	0	0	0	0	0	0	0	0	0	0	0	0
NOP	00011100		101	0	0	0	0	0	0	0	0	0	0	0	0
JACCNEG	00011101		010	0	0	0	0	0	0	0	**1**	**1**	**0**	**0**	**0**
	00011110		111	0	0	0	0	0	0	0	0	0	0	0	0

μ-Instruction Format

μ Opcode	Control signals/Next Address
3 bits	12 bits

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	If $IR_0 = 1$ then μPC ← Next Addr
100	If $IR_1 = 1$ then μPC ← Next Addr
101	If $IR_2 = 1$ then μPC ← Next Addr
110	Not Used
111	μPC ← Next Address

Instruction Format

OPCODE	Address
IR_0 IR_1 IR_2	IR_3 IR_4 IR_5 IR_6 IR_7

OPCODE	Mnemonic
0 0 0	LDACC
0 0 1	STACC
0 1 0	ADD
0 1 1	SUB
1 0 0	JUMP
1 0 1	JACCNZ
1 1 0	JACCNEG
1 1 1	NOP

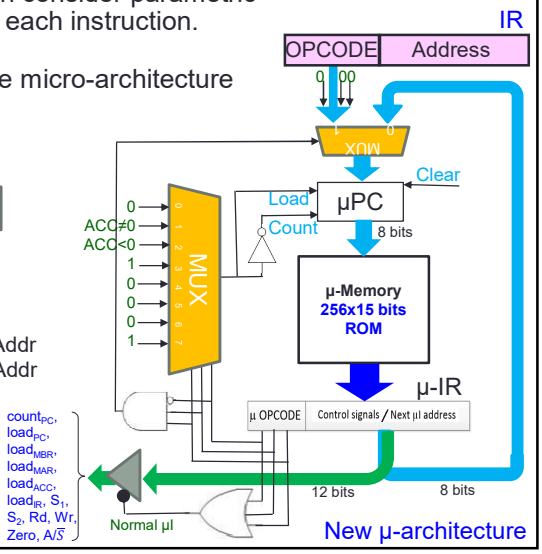
Control Unit: Microprogrammed CU

In order to reduce the hassle of checking OPCODE's bits and complex μ-programming, we can consider parametric branches to execution phase of each instruction.

To do so, we need to change the micro-architecture and micro-instruction format.

New μ-Instruction Format

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	μPC ← Ext. Addr: $IR_0IR_1IR_200$
100	Not Used
101	Not Used
110	Not Used
111	μPC ← Next Address



Control Unit: Microprogrammed CU

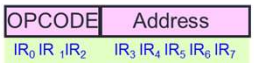
Now, with a small change in OPCODE assignment we can rewrite the μ-program to be executed by the new μ-architecture as follows:

	μ Address	μ Instruction	μOpc	c_{PC}	L_{PC}	L_{MBR}	L_{MAR}	L_{ACC}	L_{IR}	S_1	S_2	R	W	Z	$A/\overline{S}$
NOP	00000000		000	0	0	0	1	0	0	1	1	0	0	0	0
FETCH	00000001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000010		000	1	0	0	0	0	1	0	0	0	0	0	0
	00000011		011	0	0	0	0	0	0	0	0	0	0	0	0
LDACC	00000100		000	0	0	0	1	0	0	0	0	0	0	0	0
	00000101		000	0	0	1	0	0	0	0	0	1	0	0	0
	00000110		000	0	0	0	0	1	0	0	0	0	0	0	1
	00000111		111	0	0	0	0	0	0	0	0	0	0	0	0
STACC	00001000		000	0	0	1	1	0	0	0	0	0	0	0	0
	00001001		000	0	0	0	0	0	0	0	0	0	1	0	0
	00001010		111	0	0	0	0	0	0	0	0	0	0	0	0
	00001011		000	0	0	0	0	0	0	0	0	0	0	0	0
ADD	00001100		000	0	0	0	1	0	0	0	0	0	0	0	0
	00001101		000	0	0	1	0	0	0	0	0	1	0	0	0
	00001110		000	0	0	0	0	1	0	0	0	0	0	1	1
	00001111		111	0	0	0	0	0	0	0	0	0	0	0	0
SUB	00010000		000	0	0	0	1	0	0	0	0	0	0	0	0
	00010001		000	0	0	1	0	0	0	0	0	1	0	0	0
	00010010		000	0	0	0	0	1	0	0	0	0	0	1	0
	00010011		111	0	0	0	0	0	0	0	0	0	0	0	0
JUMP	00010100		000	0	1	0	0	0	0	0	0	0	0	0	0
	00010101		111	0	0	0	0	0	0	0	0	0	0	0	0
	00010110		000	0	0	0	0	0	0	0	0	0	0	0	0
	00010111		000	0	0	0	0	0	0	0	0	0	0	0	0
JACCNZ	00011000		001	0	0	0	0	0	0	0	1	0	1	0	0
	00011001		111	0	0	0	0	0	0	0	0	0	0	0	0
	00011010		000	0	0	0	0	0	0	0	0	0	0	0	0
	00011011		000	0	0	0	0	0	0	0	0	0	0	0	0
JACCNEG	00011100		010	0	0	0	0	0	0	0	1	0	1	0	0
	00011101		111	0	0	0	0	0	0	0	0	0	0	0	0

New μ-Instruction Format

μOpc	Operation
000	Normal μI
001	If (ACC)≠0 then μPC ← Next Addr
010	If (ACC)<0 then μPC ← Next Addr
011	μPC ← External Address
100	Not Used
101	Not Used
110	Not Used
111	μPC ← Next Address

Instruction Format



OPCODE	Mnemonic
0 0 0	NOP
0 0 1	LDACC
0 1 0	STACC
0 1 1	ADD
1 0 0	SUB
1 0 1	JUMP
1 1 0	JACCNZ
1 1 1	JACCNEG

Control Unit: Microprogrammed CU

Reducing micro-memory capacity

One important issue with micro-programmed systems is the latency of micro-memory that in turn will reduce the whole CPU speed.

This is because the latency (also cost and power consumption) of micro-memory tightly depends to its size.

The smaller the micro-memory,
the better implementation under area limitation, and hence
the higher ultimate speed and lower power consumption and cost.

There are two main techniques to reduce the size of micro-memory:

1. Vertical micro-instructions
2. Nano-memory

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ-instructions

Looking at the used patterns of control signals in a micro-program, one can easily see that most bits are 0 and few bits are 1. This is because when a specific micro-instruction (like data transfer or computation boxes in the flowchart) is executed few components are involved and only the control signals associated to them are activated.

This means we can properly partition the control signals into groups where in each group at most one control signal is active in all used patterns. We can then encode each group's pattern with a short code that show the position of the 1 in the group's pattern.

If the length of a group's pattern is k bits, only $\log_2(k+1)$ bits are needed to encode the group's pattern.
→ We need some decoder (latency overhead) to decode codes and get the actual signals. ☹

Group bits positions → 1 2 3 4 5 6 … k
Encoded by 00...00 for 0 0 0 0 0 0 … 0
Encoded by 00...01 for 1 0 0 0 0 0 … 0
Encoded by 00...10 for 0 1 0 0 0 0 … 0
…..
Encoded by 11...11 for 0 0 0 0 0 0 … 1

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ-instructions

Example: Consider the micro-programmed computer.

The control signals can be grouped as shown where at most one bit 1 appears in each group.

Group 1 can be coded by 3 bits and Group 2 and Group 3 each can be coded by 2 bits.
→ We will have 7 bits instead of 12 bits (~40%).

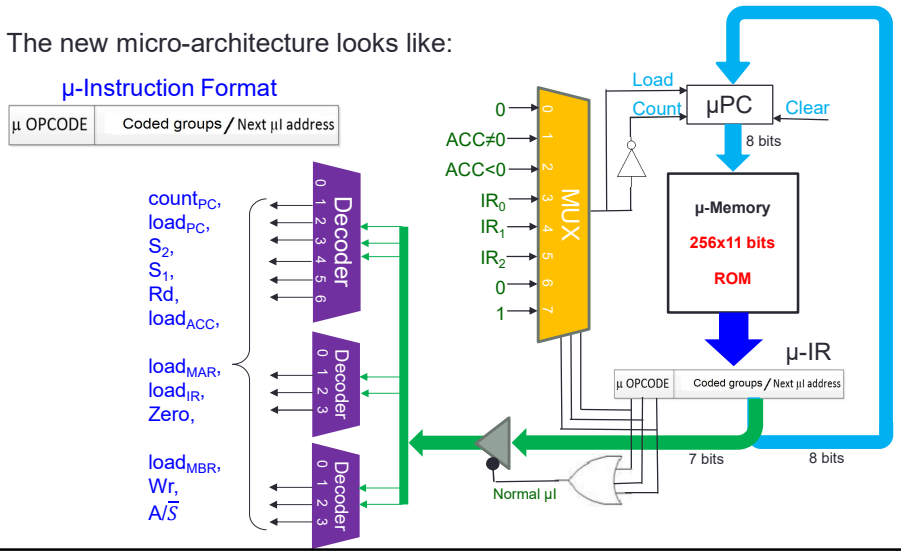
In real processors with hundreds of control signals saving can be big, say 80% ☺

C_{PC}	L_{PC}	S_1	L_{ACC}	R	W	L_{MAR}	L_{IR}	Z	S_2	$A/\overline{S}$	L_{MBR}
0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0	1	0
0	1	0	0	0	0	0	0	0	0	0	0
Group 1						Group 2			Group 3		

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Vertical μ-instructions

The new micro-architecture looks like:



Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

Looking at the used patterns of control signals in a micro-program, one can easily see that different used patterns are a limited number.
This is because most micro-instructions may be reused in different places of the flowchart.

This means we can store such different patterns in a special memory called Nano-memory and use the patterns' addressed instead of the patterns themselves.

If the number of different used patterns is k (out of theoretical 2^n possible patterns of n signals), only $\log_2 k$ bits are needed to address these k patterns in the Nano-memory.

→ A very big saving (in real systems, it can be more than 90% ☺
→ The latency overhead of accessing the added Nano-Memory ☹

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

Example: Consider the micro-programmed computer.

The control signals use only 10 different patterns.

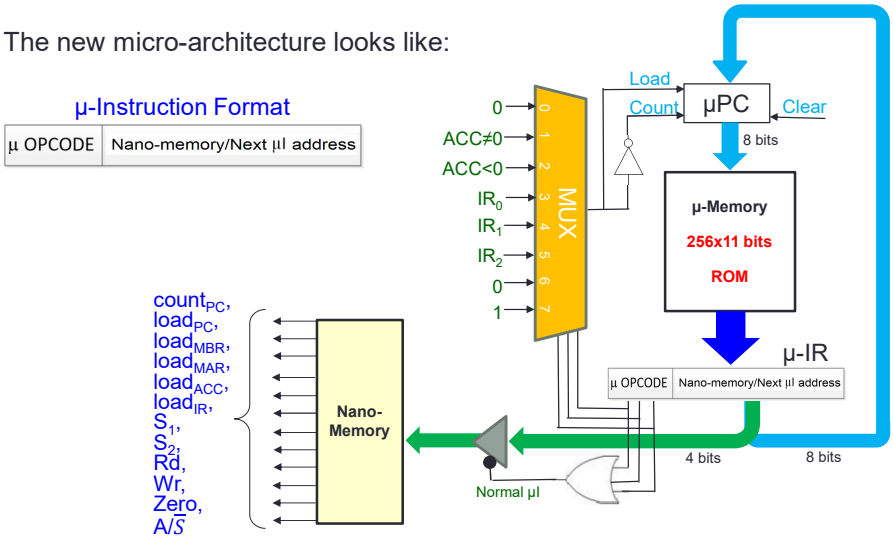
→ We can keep them in a Nano-memory addressed by 4 bits.
That is %75 savings ☺

C_{PC}	L_{PC}	S_1	L_{ACC}	R	W	L_{MAR}	L_{IR}	Z	S_2	$A/\overline{S}$	L_{MBR}
0	0	1	0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1
1	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	0	1	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0

Control Unit: Microprogrammed CU

Reducing micro-memory capacity: Nano-Memory

The new micro-architecture looks like:



Control Unit: Microprogrammed CU

Reducing micro-memory capacity:

Note that in both Vertical μ-Instruction technique and Nano-memory technique, the Control Signals field is replaced with a bit pattern whose length is the maximum of the resulted pattern length and the address length in branch micro-instructions (or number of bits addressing the μ-memory).

This is because the address bit patterns of branch μ-instructions do not comply with the philosophy behind Vertical μ-Instruction technique and Nano-memory technique. So, we must keep the address patterns unchanged.

END OF SLIDES

QUESTIONS?